MITSUBISHI ELECTRIC
Changes for the Better

for a greener tomorrow

FX5U PLC
Basic & Application Instructions

Training Department
Factory Automation Engineering Centre
Factory Automation & Industrial Division
Mitsubishi Electric India Private Limited
MEI-FAID-FATRAINING@asia.meap.com

MITSUBISHI ELECTRIC INDIA

COPY RIGHT : 2011 MITSUBISHI ELECTRIC INDIA PVT. LTD. ALL RIGHTS RESERVED

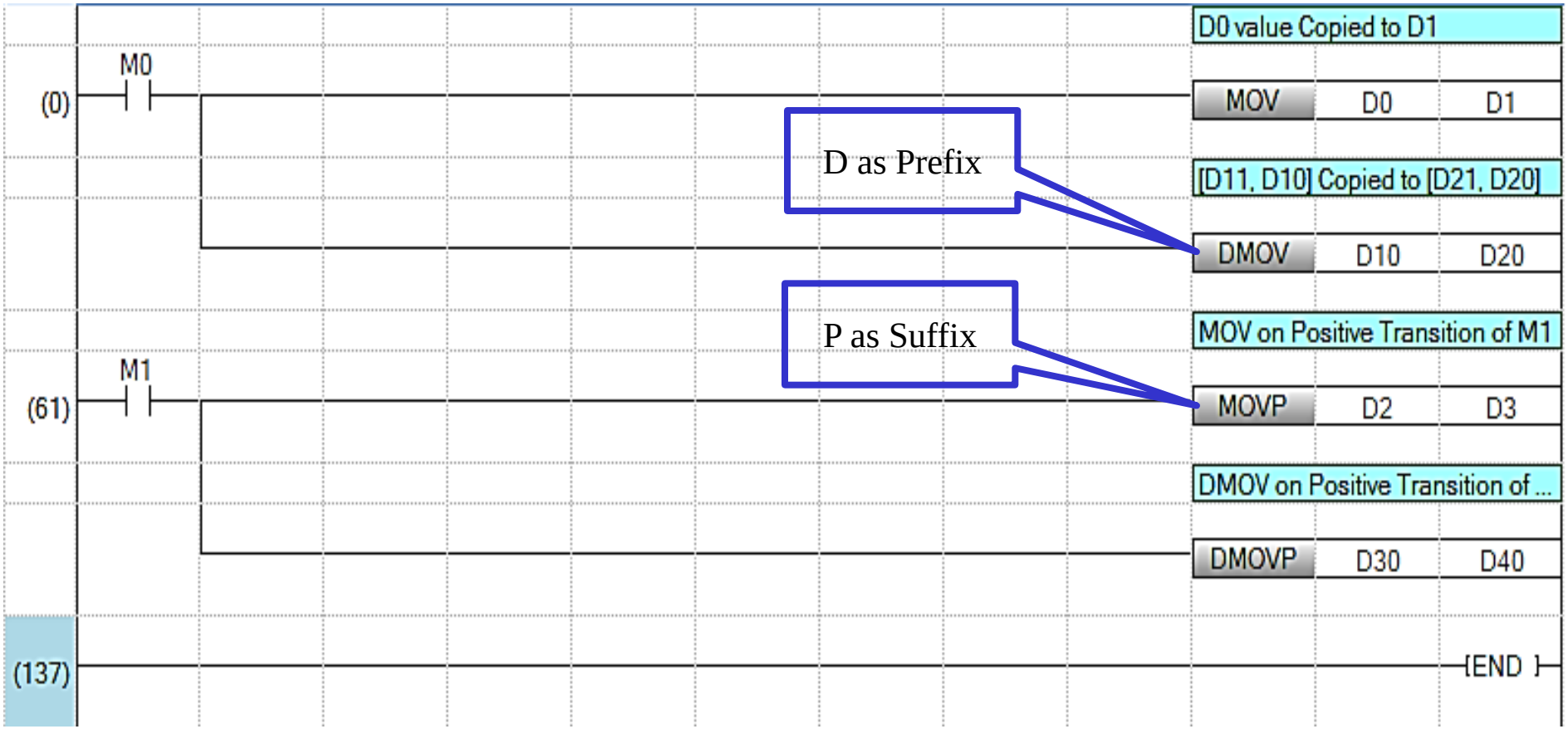
Contents

- Structure of Instructions
- Comparison Instructions
- Arithmetic Operation Instructions
- Logical operation Instruction
- Data Conversion Instructions
- Data Transfer Instructions
- Grouping of Bit Devices
- Indexing of Bits and Words
- Program Branch Instructions
- Structuring Instructions

Structure of Instructions

- Arguments of instructions are
  - word devices,
  - group of bit devices
  - Constants
- Instruction has FOUR forms
  - No prefix and suffix
  - P as a suffix
  - D as a prefix
  - P and D both Simultaneously
- P as a suffix : It works on the up transition of input logic
- D as a prefix : It works for double words
- Same instruction works with two and three arguments

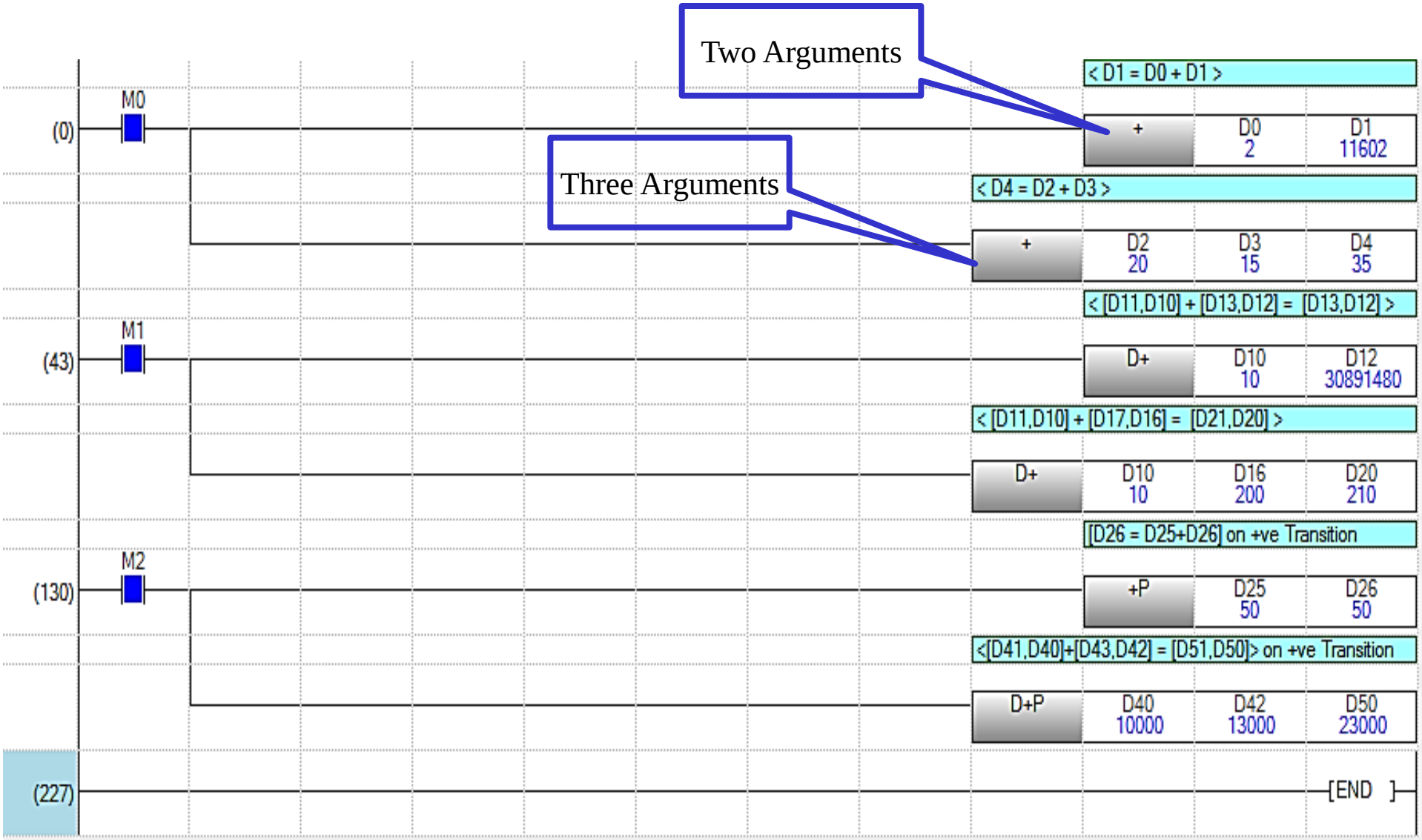
Example of P and D



Two or Three Arguments

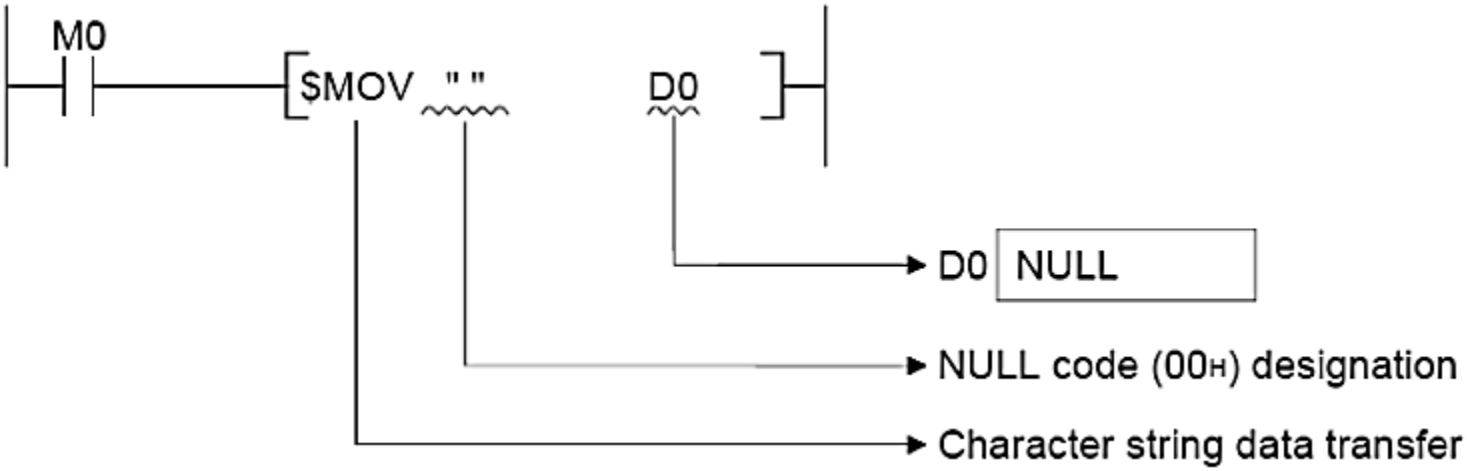
- Generally used with Math and Logical instructions
- Math and Logical instructions need Sources and Destination
- Two different conditions
  - Sources and destination are different
  - One of the sources is used as a destination
- Source may be word device, constants, Group of bit devices
- Destination must not be a constant
- If sources and destinations are different, three arguments are required
- If one of the sources is used as a destination then two arguments are required

Two or Three Arguments



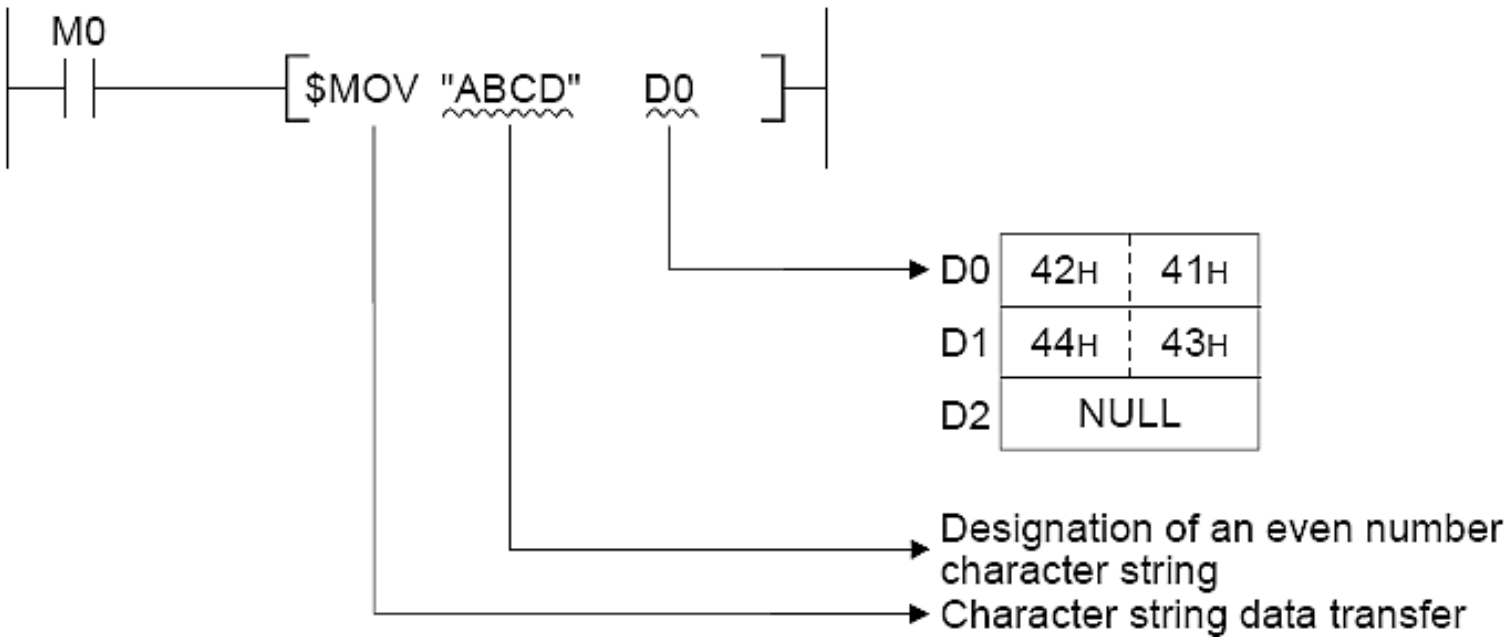
Handling Strings

- Characters are stored as its ASCII value in register
- One character needs one byte
- Two characters are stored in single 16 bit register
- NULL code (00H) is concatenated with string to indicate end of it
- If character string is NULL it takes one word



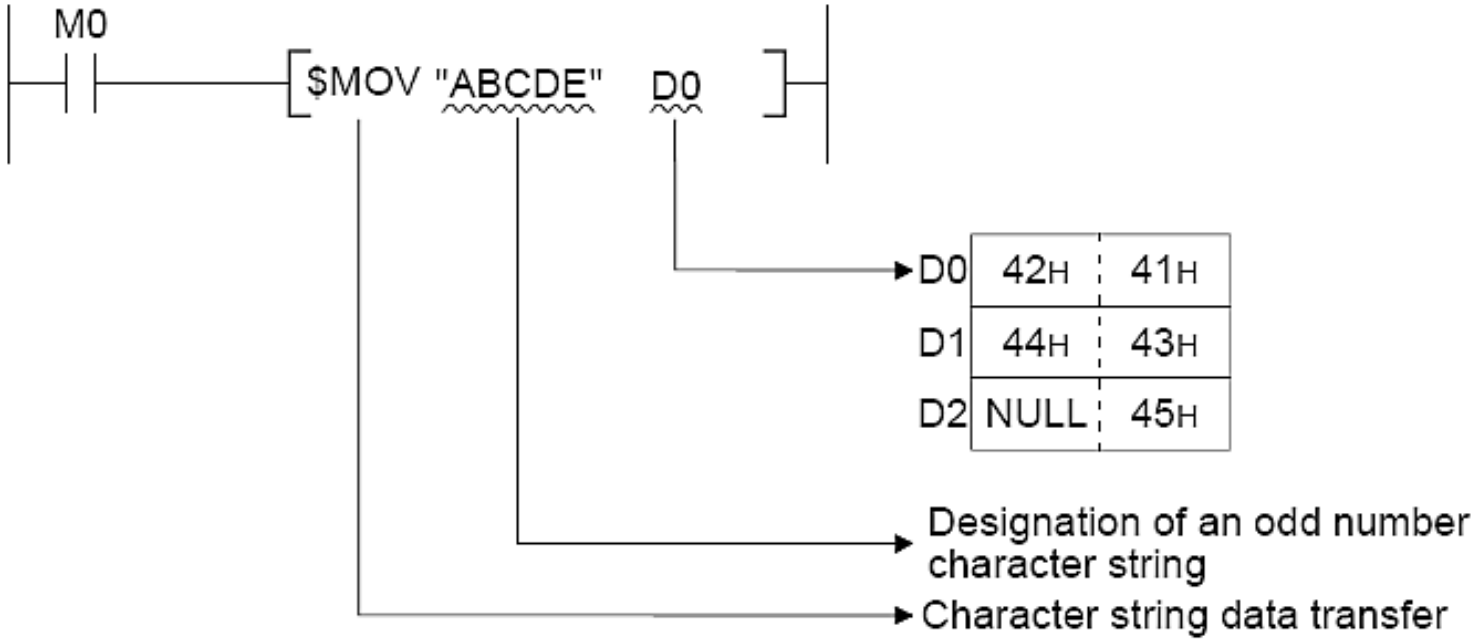
Handling Strings

- When character string is even
  - Uses (number of characters/2 + 1) words, and stores character string and NULL code
  - For example, if "ABCD" is transferred to D0, the character string ABCD is stored at D0 and D1, and the NULL code is stored at D2



Handling Strings

- When number of characters is odd
  - Uses (number of characters/2) words (rounds up decimal fractions) and stores the character string and NULL code
  - For example, if "ABCDE" is transferred to D0, the character string (ABCDE) and the NULL code are stored from D0 to D2



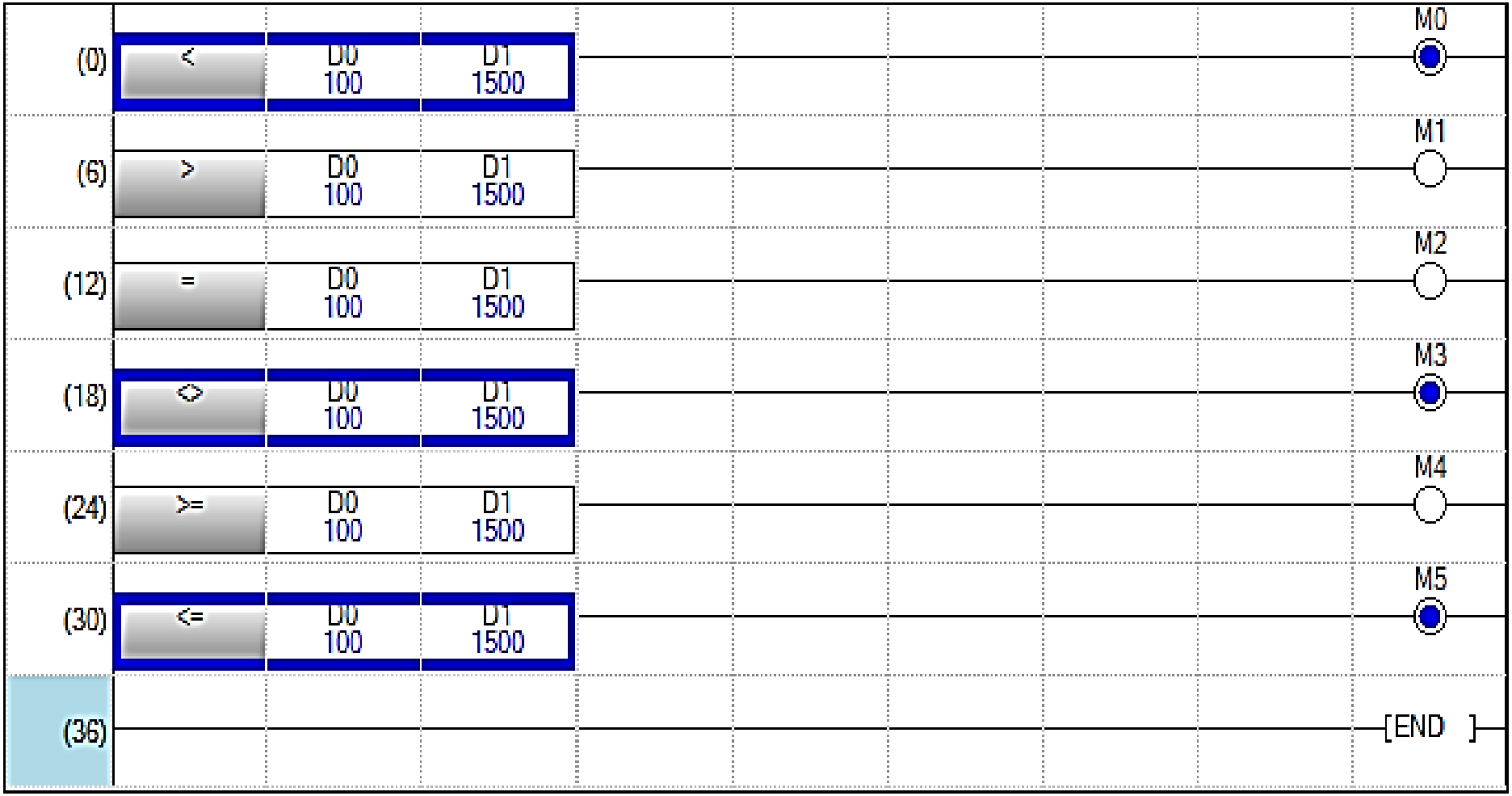
Comparison Instructions

- Instruction compares two data and results in true or false condition
- It can be used with other sequence instructions
- All instructions are input instructions except BKCMP which is an output instruction
- Compare instructions are available to compare
  - 16 bit data — Comparison condition — e.g. LD=
  - 32 bit data — D and comparison condition — e.g. LDD=
  - Real numbers — E and comparison condition — e.g. LDE=
  - String data — $ and comparison condition — e.g. LD$=
- Block of 16 bit data comparison (BKCMP)

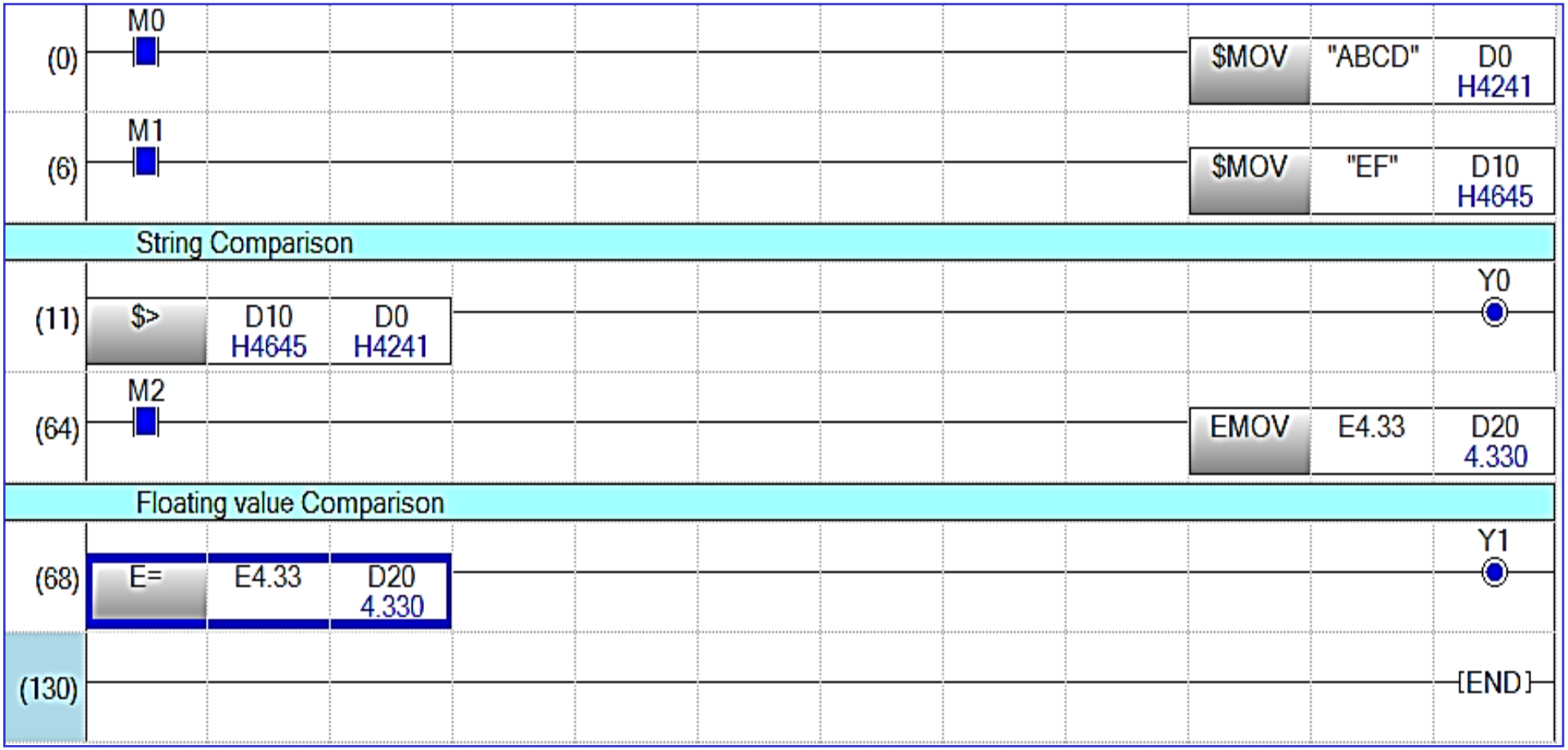
Compare Instructions

- Basic comparisons
  - LD= : Equal to
  - LD> : Greater than
  - LD< : Less than
  - LD<> : Not equal to
  - LD<= : Less than or equal to
  - LD>= : Greater than or equal to
- Double word comparison is available for all shown above (e.g. LDD=/LDD<)
- Pulsed variations is not available

Example of Comparison

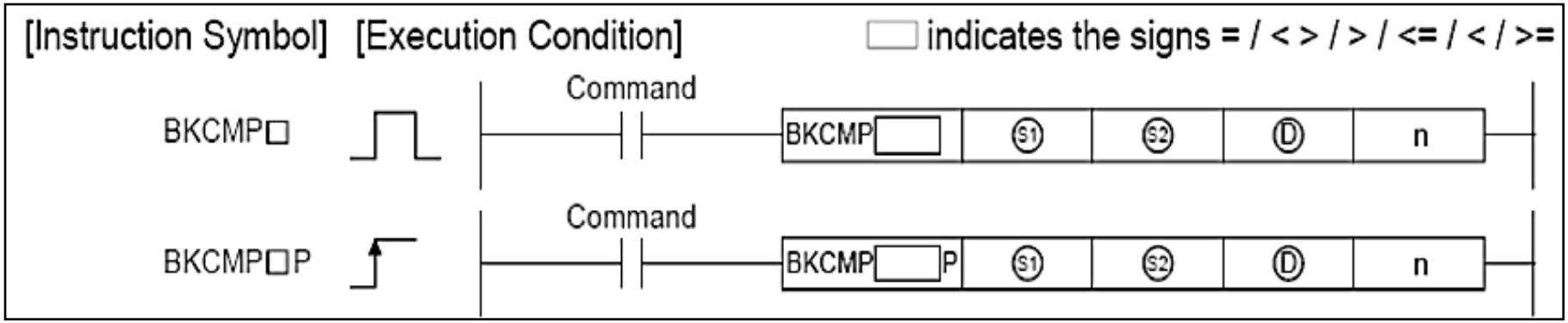


Example of Comparison



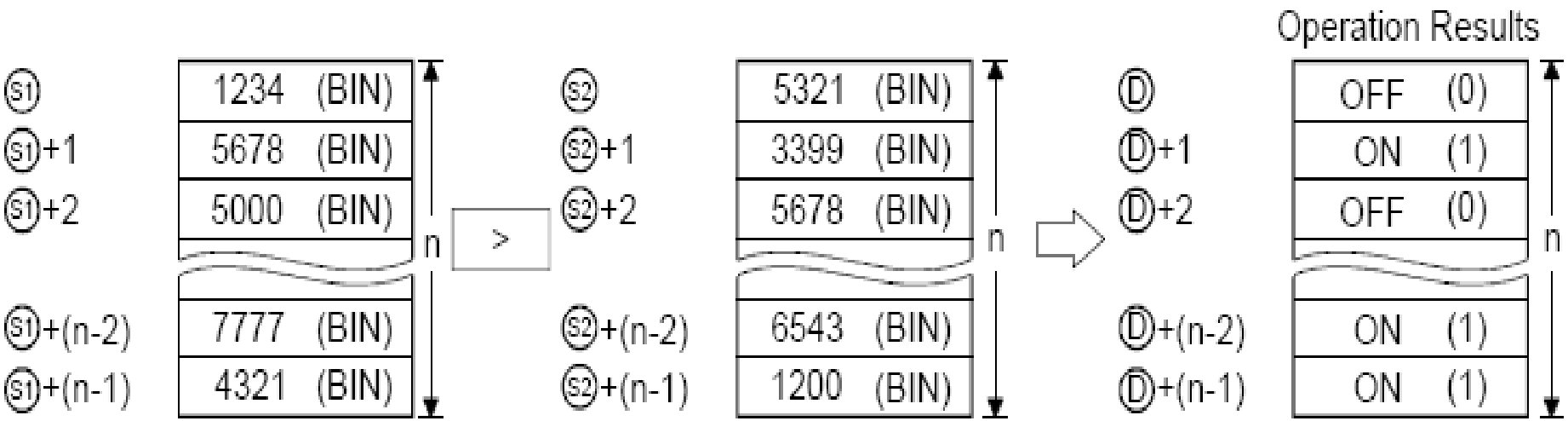
Block Compare Instruction

- BKCMP compares two blocks of 16 bit data and results in On/Off of bit devices
- If the comparison condition has been met, the device designated by D will be turned ON otherwise OFF



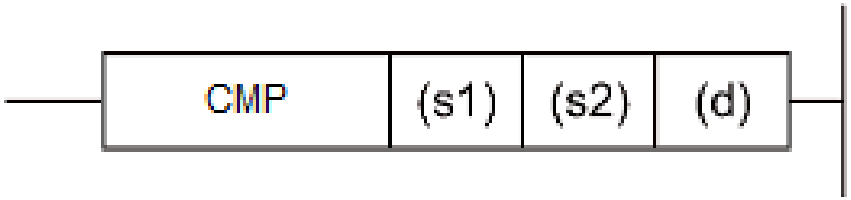
Block Compare Instruction

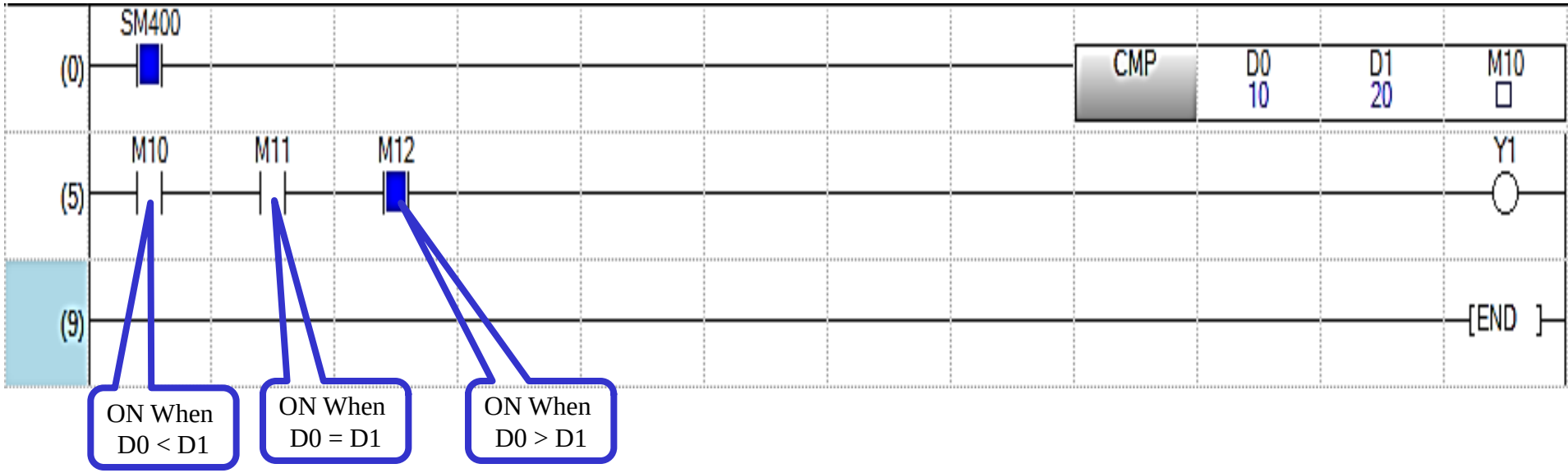
- BKCMP compares two blocks of 16 bit data and results in On/Off of bit devices
- If the comparison condition has been met, the device designated by D will be turned ON otherwise OFF



Compare by CMP instruction

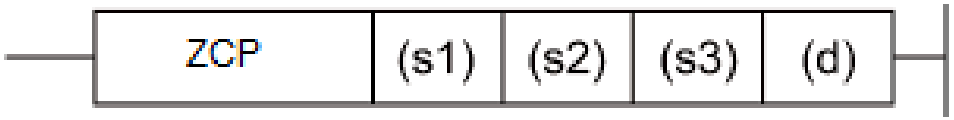
- CMP instruction is Used to Compare two 16 bit Binary data and Sets the bit according to result (small, equal, large).
- It is the output instruction so an input signal is required to Enable the CMP instruction.
  - s1: Comparison Data1
  - s2: Comparison Data2
  - d : Start address where the comparison result is stored



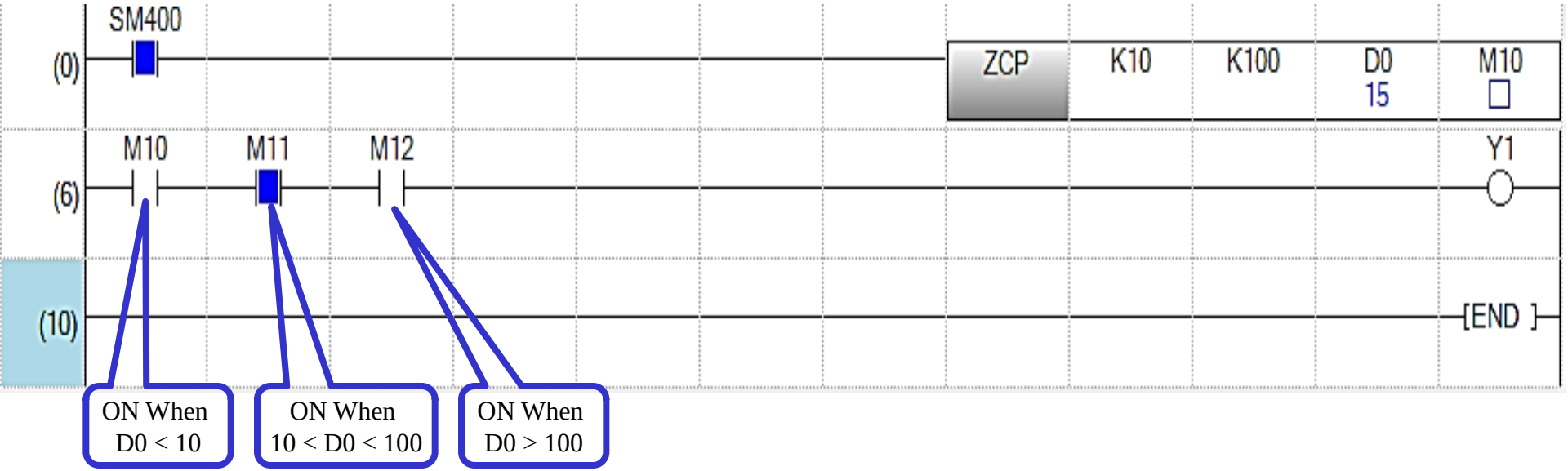


Compare within a range using ZCP instruction

- ZCP instruction compares a 16 bit value within a specified range (low limit and high limit) and results output according to condition matched (lower, within range, higher)



  - s1: lower Limit Value
  - s2: Higher Limit Value
  - s3: Comparison data
  - d : Start device where the comparison result is stored



Arithmetic Instructions

- 16 bit Addition and Subtraction

Category	Instruction Symbols	Symbol	Processing Details	Execution Condition
BIN 16-bit addition and subtraction operations	+	+ S D	• (D)+(S) → (D)	
	+P	+P S D		
	+	+ S1 S2 D	• (S1)+(S2) → (D)	
	+P	+P S1 S2 D		
	-	− S D	• (D) - (S) → (D)	
	-P	−P S D		
	-	− S1 S2 D	• (S1) - (S2) → (D)	
	-P	−P S1 S2 D		

Remember
D= D - S

Arithmetic Instructions

- 32 bit Addition and Subtraction

Category	Instruction Symbols	Symbol	Processing Details	Execution Condition
BIN 32-bit addition and subtraction operations	D+	D+ S D	• (D+1, D)+(S+1, S) → (D+1, D)	
	D+P	D+P S D		
	D+	D+ S1 S2 D	• (S1+1, S1)+(S2+1, S2) → (D+1, D)	
	D+P	D+P S1 S2 D		
	D-	D− S D	• (D+1, D)-(S+1, S) → (D+1, D)	
	D-P	D−P S D		
	D-	D− S1 S2 D	• (S1+1, S1)-(S2+1, S2) → (D+1, D)	
	D-P	D−P S1 S2 D		

Arithmetic Multiplication & Division

- 16 bit /32 bit Multiplications and Divisions

Category	Instruction Symbols	Symbol	Processing Details	Execution Condition
BIN 16-bit multi tion divis oper	*	* S1 S2 D	• (S1) * (S2) → (D+1, D)	
		P D		
		S1 S2 D	• (S1)/(S2) → Quotient(D), Remainder (D+1)	
		P S1 S2 D		
BIN 32-bit multiplica-tion and division operations	D*	D* S1 S2 D	• (S1+1, S1) * (S2+1, S2) → (D+3, D+2, D+1, D)	
	D*P	D*P S1 S2 D		
	D/		+1, S1)/(S2+1, S2) → Quotient (D+1, D), Remainder (D+3, D+2)	
	D/P			

Result in two consecutive registers, MSB in last register

Result in four consecutive registers, MSB in last register

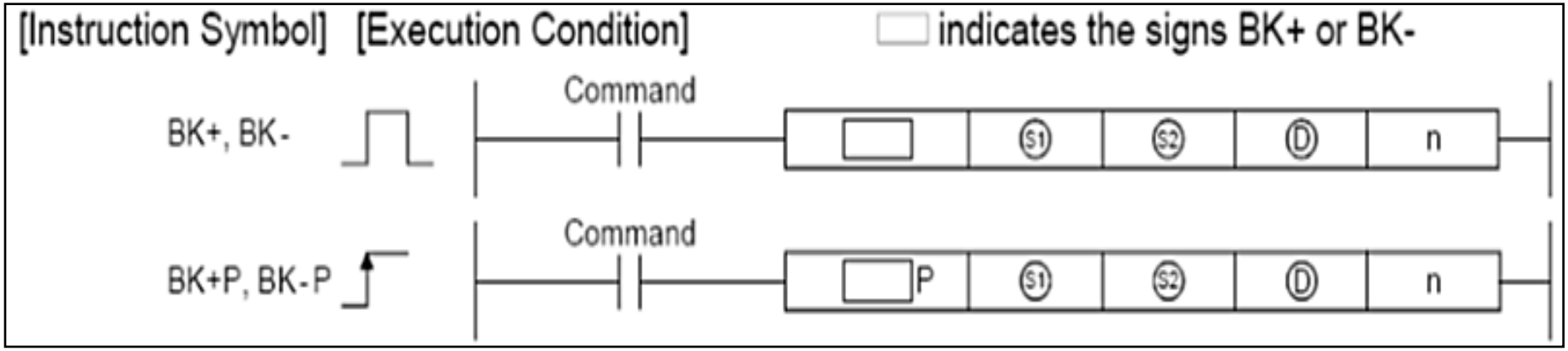
Arithmetic Instruction

- Increment / Decrement Instructions

Category	Instruction Symbols	Symbol	Processing Details	Execution Condition
BIN data increment	INC	INC D	• (D)+1 → (D)	
	INCP	INCP D		
	DINC	DINC D	• (D+1, D)+1 → (D+1, D)	
	DINCP	DINCP D		
	DEC	DEC D	• (D)-1 → (D)	
	DECP	DECP D		
	DDEC	DDEC D	• (D+1, D)-1 → (D+1, D)	
	DDECP	DDECP D		

Block Addition/ Subtraction

- Adds n-points of BIN (16 bit) data from the device designated by S1 and n-points of BIN data from the device designated by S2 and stores the result from the device designated by D onward



Data Conversion Device Monitor

1 [Device/Buffer Memory Batch Monitor] Monitoring

Device Name D0 | Detailed Conditions | Monitoring

Buffer Memory Unit (HEX) Address DEC | Stop Monitoring

Device Name	F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0	Current Value	String
D0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	256	..
D1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	..
D2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	..
D3	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	1280	..
D4	0	0	0	0	0	0	0	0	0	1	1	0	0	1	0	0	100	d.
D5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	..
D6	0	1	0	0	1	0	1	1	0	1	0	0	0	0	0	0	19264	@K
D7	0	0	0	0	0	0	0	0	0	1	0	0	1	1	0	0	76	L.
D8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	..
D9	0	1	0	0	0	0	1	1	0	1	0	0	1	0	0	0	17224	HC
D10	1	0	0	1	0	1	1	0	1	0	0	0	0	0	0	0	-27008	€-
D11	0	1	0	0	1	0	1	0	1	0	0	1	1	0	0	0	19096	˜J
D12	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	10	..
D13	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	..
D14	0	0	0	0	0	1	1	1	1	1	0	1	0	0	0	0	2000	Đ.
D15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	..
D16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	..
D17	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	..
D18	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	10	..
D19	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	..
D20	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	10	..
D21	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	..
D22	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	..
D23	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	..
D24	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	..
D25	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	..

Data Transfer Instructions

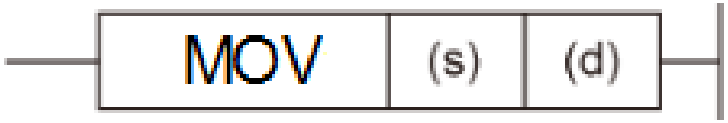
Various types of data transfer Instructions as Follows

1. 16 bit and 32 bit data, as an integer
   - MOV, MOVP, DMOV, DMOVP
2. Floating data (2 registers, 32 bit representation of floating data)
   - EMOV, EMOVP
3. String data
   - $MOV, $MOVP
4. Block movement
   - BMOV and BMOVP
5. DATA Exchange Instructions
   - XCH, XCHP, DXCH, DXCHP
6. One Source copied to multiple places
   - FMOV, FMOVP

Data Transfer Instructions

MOV(P):

- This instruction transfer the 16-bit binary data in the device specified by (s) to the device specified by (d).

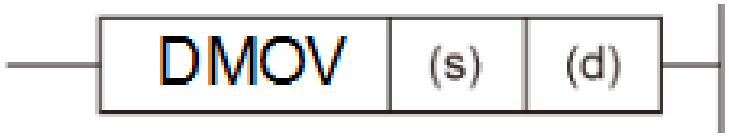


Operand	Description	Range	Data type
(s)	Transfer source data or device number for storing data	-32768 to +32767	16-bit signed binary
(d)	Transfer destination device number	—	16-bit signed binary

Data Transfer Instructions

DMOV(P):

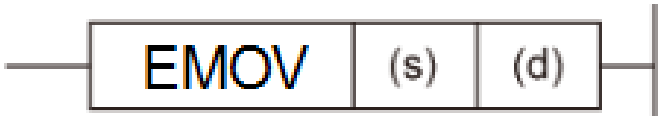
- These instructions transfer the 32-bit binary data in the device specified by (s) to the device specified by (d).



Operand	Description	Range	Data type
(s)	Transfer source data or device number for storing data	-2147483648 to +2147483647	32-bit signed binary
(d)	Transfer destination device number	—	32-bit signed binary

EMOV(P)/DEMOV(P)

- These instructions transfer the single-precision real number data stored in the device specified by (s) to the device specified by (d).
- The EMOV(P) instructions can also be used as DEMOV(P).

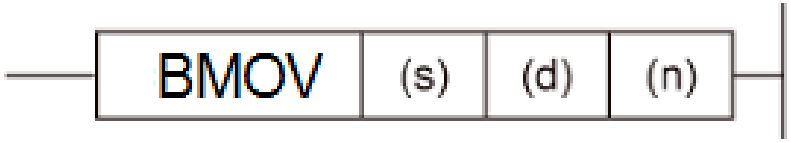


Operand	Description	Range	Data type
(s)	Data to be transferred or head device number where the data to be transferred is stored	$0, 2^{-126} < \|(s)\| < 2^{128}$	Single-precision real number
(d)	Device number storing the data in transfer destination	—	Single-precision real number

Data Transfer Instructions

BMOV(P):

- These instructions block transfer the 16-bit binary data of (n) number of devices starting from the one specified by (s) to the device specified by (d).

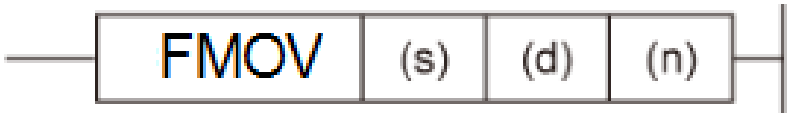


Operand	Description	Range	Data type
(s)	Head device for storing the data to be transferred	—	16-bit signed binary/32-bit signed binary
(d)	Head number of the transfer-destination device	—	16-bit signed binary/32-bit signed binary
(n)	Number of transfers	1 to 65535	16-bit unsigned binary

Data Transfer Instructions

FMOV(P) :

- These instructions transfer (n) point(s) of data identical to the 16-bit binary data in the device specified by (s) to the devices specified by (d).



Operand	Description	Range	Data type
(s)	Data to be transferred or the head device for storing the data to be transferred	-32748 to +32767	16-bit signed binary
(d)	Head device of the transfer-destination	—	16-bit signed binary
(n)	Number of transfers	1 to 65535	16-bit unsigned binary

Data Transfer Instructions

DFMOV(P) :

These instructions transfer (n) point(s) of data identical to the 32-bit binary data in the device specified by (s) to the devices specified by (d).

Operand	Description	Range	Data type
(s)	Data to be transferred or the head device for storing the data to be transferred	-2147483648 to +2147483647	32-bit signed binary
(d)	Head device of the transfer-destination	—	32-bit signed binary
(n)	Number of transfers	1 to 65535	16-bit unsigned binary

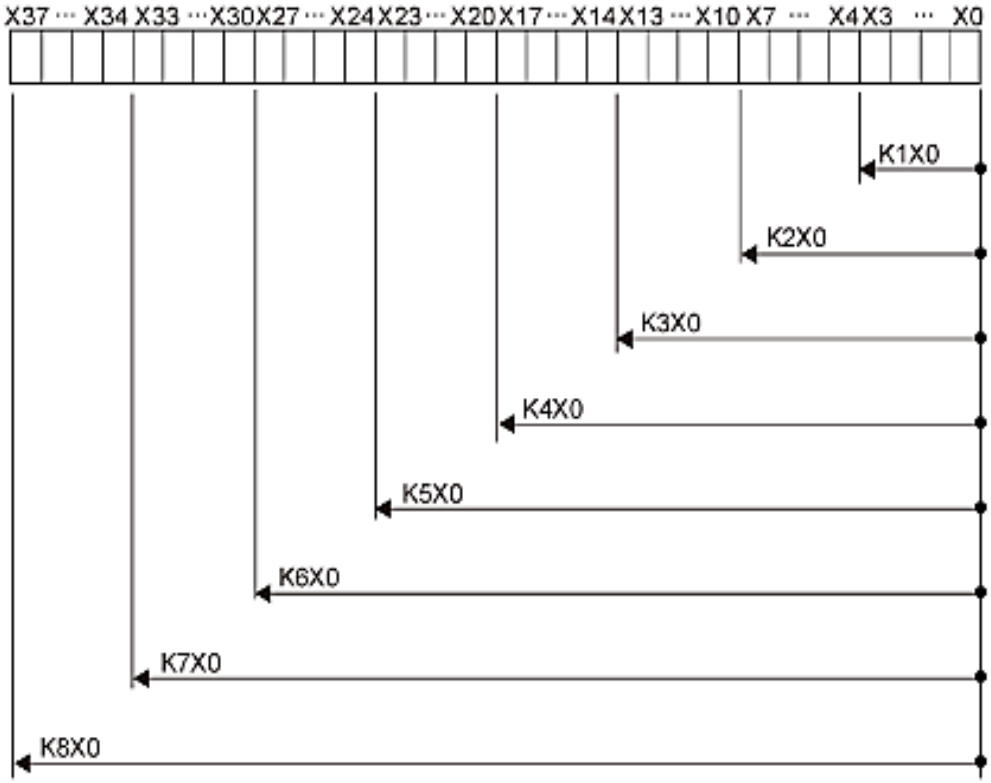
Example of Data Transfer Instructions

Grouping of Bit Devices

- It is used to process group of bit devices as a word devices
- Examples
  - To store status of discrete inputs into D registers, x20 to x2F
  - To start group of output devices, Y0 to Y7
  - Update internal bits M0 to M15 from D3
- Grouping of continuous range of bit devices only
- Grouping in terms of Nibble and Multiple of Nibble (4bit)
- Maximum 32 bit devices can be grouped (8 Nibbles)
- Grouped bit devices are used as an argument in word type instructions like math or logical instructions

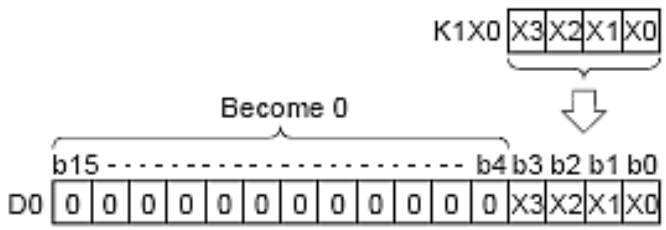
Grouping of Bit Devices

- KnD#
  - n is a no. of Nibbles of D type
  - n can be 1 to 8
  - # starting address
  - End address depends on Device type
  - Decimal or Hexadecimal addressing
- Examples
  - K1X0 : X0 to X3
  - K2M0 : M0 to M7
- K3Y20 : Y20 to Y2B
  - K2M13 : M13 to M20



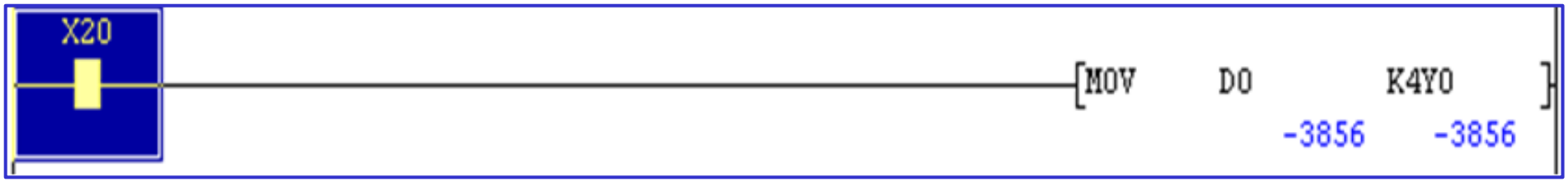
Remember : Addressing of M is in decimal

Example of Grouping

Ladder Example	Processing
With 16-bit instruction X010 — [MOV K1X0 D0] Source Ⓢ data	
When source Ⓢ data is a numerical value X010 — [MOV H1234 K2M0] Destination Ⓓ	
When source Ⓢ data is a word device X10 — [MOV D0 K2M100] Destination Ⓓ	

Example of Grouping

- Content of D0 is to be transferred to Y0 to Y0F when X20 is on
- Key points
  - Sixteen bits are to be grouped
  - K4 combines sixteen bit devices
  - K4Y0 is used

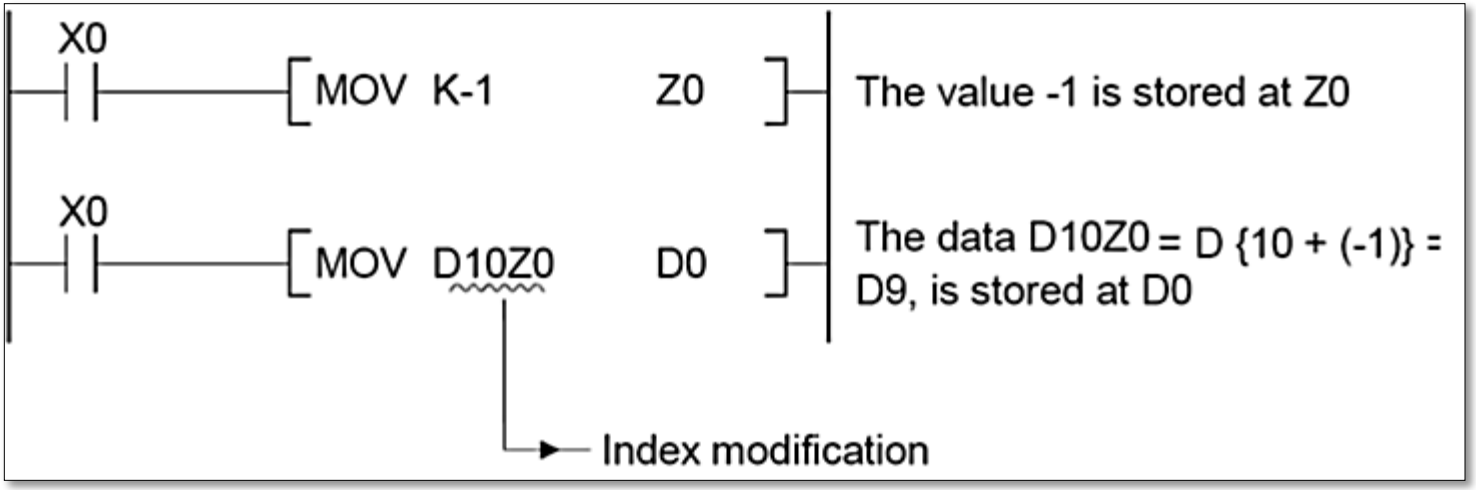


Device	+F E D C	+B A 9 8	+7 6 5 4	+3 2 1 0
D0	1 1 1 1	0 0 0 0	1 1 1 1	0 0 0 0
D1	0 0 0 1	0 0 1 1	1 1 1 0	1 1 0 0

Device	+F E D C	+B A 9 8	+7 6 5 4	+3 2 1 0
Y0	1 1 1 1	0 0 0 0	1 1 1 1	0 0 0 0
Y10	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0

Indexing Modification

- It is an indirect setting made by using an index register
- When it is used in a sequence program, the device to be used will become the device number designated directly plus the contents of the index register
- For example, if D2Z2 has been designated, the designated device is calculated as follows:
  - If Z2 is 3 then D(2+3)=D5 becomes the designated device
- There are 16 index registers, from Z0 to Z15
- Each index register can be set between -32768 and 32767

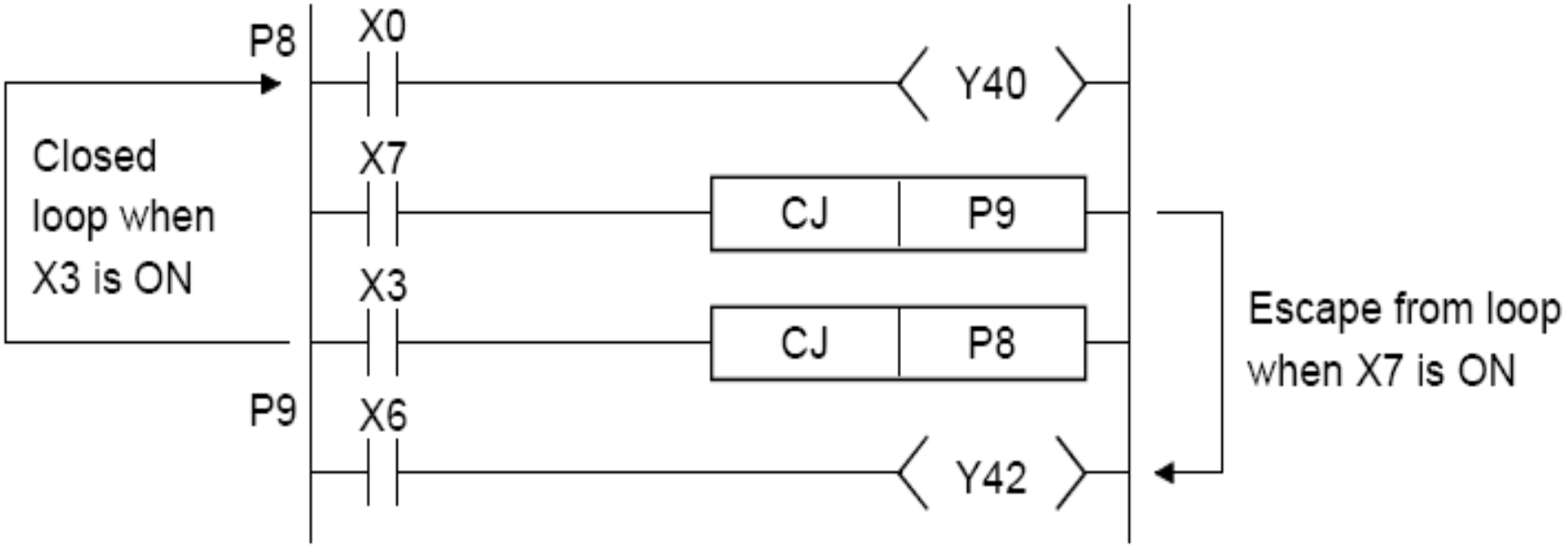


Example of Indexing

Ladder Example	Actual Process Device
X0 — MOV K20 Z0 MOV K-5 Z1 X1 — MOV K100Z0 W53Z1	X1 — MOV K120 W04E Description K100Z0....K (100+20) = K120 W53Z1.....W (53-5) = W4E Hexadecimal number
X0 — MOV K20 Z0 MOV K-5 Z1 X1 — MOV K2X50Z0 K1M38Z1	X1 — MOV K2X64 K1M33 Description K2X50Z0....K2X (50+14) = K2X64 K20 is converted to hexadecimal K1M38Z1....K1M (38-5) = K1M33
X0 — MOV K20 Z0 MOV K-5 Z1 X1 — MOV D0Z0 K3Y12FZ1	X1 — MOV D20 K3Y12A Description D0Z0.....D (0+20) = D20 K3Y12FZ1..K3Y (12F-5) = K3Y12A Hexadecimal number

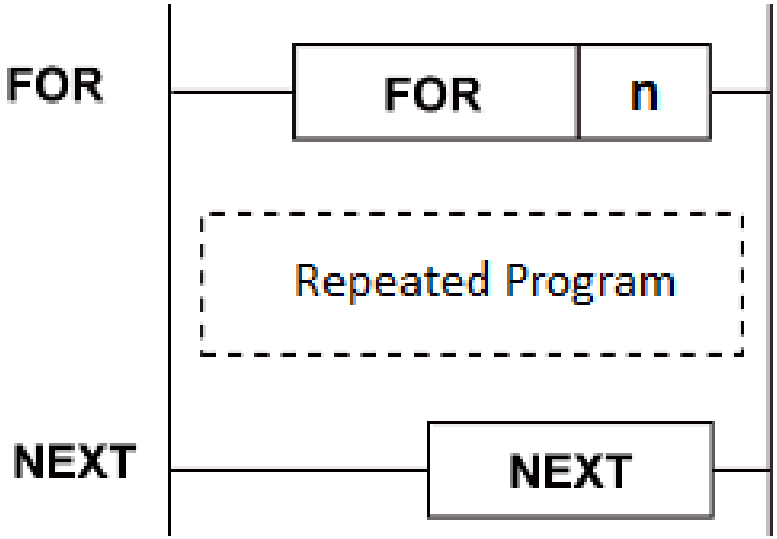
Program branch Instruction

- Program Branch Instructions
  - Conditional jump : CJ
  - Unconditional Jump : JMP
  - Jump to END : GOEND



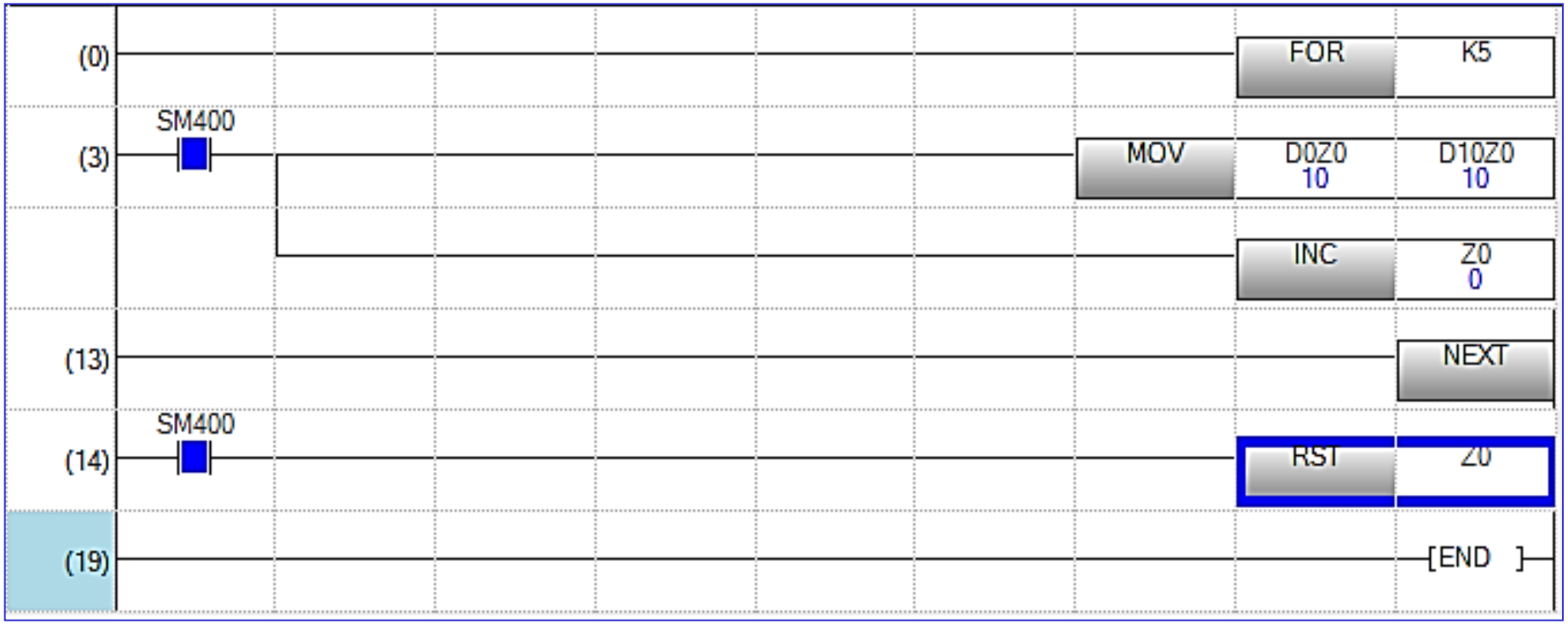
Structuring Instructions

- FOR - NEXT Instruction
  - Unconditional Loop
  - Iterations: 1 to 32767
  - For instruction can be nested up 16 levels
  - ‘n’ can be Constant or Variable



Example of FOR - NEXT

- The program executes the FOR to NEXT Instruction.
- FOR-NEXT loop is Executed 5 times in a single SCAN cycle as shown in Program.



Device Data of FOR-NEXT Example

1 [Device/Buffer Memory Batch Monitor] Monitoring

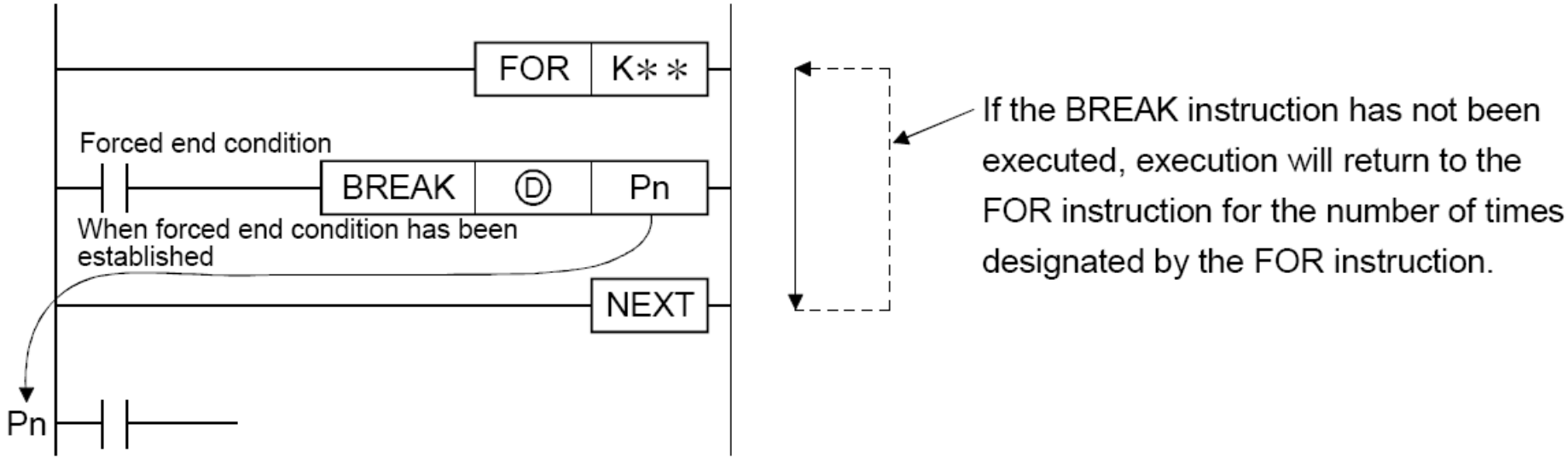
Device Name D0 — Detailed Conditions

Buffer Memory — Intelligent Module No.(U) — (HEX) — Address — DEC

Device Name	F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0	Current Value
D0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	10
D1	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	0	30
D2	0	0	0	0	0	0	0	0	0	1	1	0	0	1	0	0	100
D3	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	320
D4	0	0	0	0	0	0	0	0	1	1	1	0	0	1	1	0	230
D5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
D6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
D7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
D8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
D9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
D10	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	10
D11	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	0	30
D12	0	0	0	0	0	0	0	0	0	1	1	0	0	1	0	0	100
D13	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	320
D14	0	0	0	0	0	0	0	0	1	1	1	0	0	1	1	0	230
D15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
D16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
D17	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
D18	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

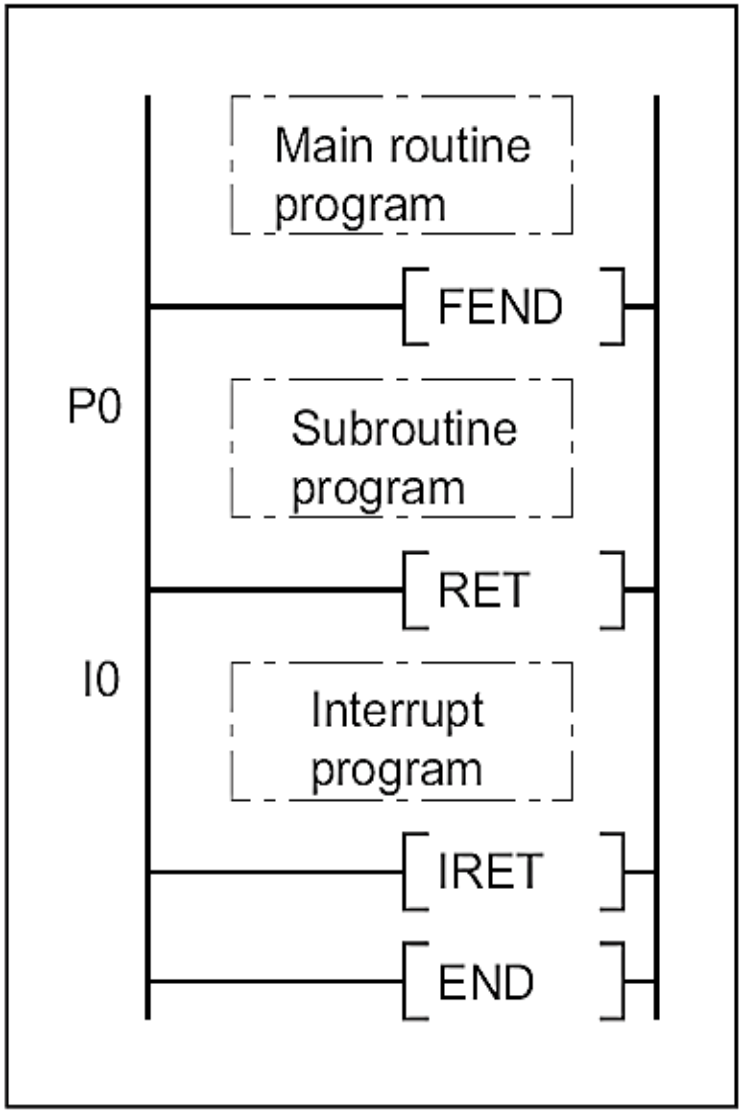
Break Instruction

- To come out of the loop prior finishing total iteration insert a BREAK instruction
- Forces the end of repetition processing based on the FOR to NEXT instruction loop, and shifts to the execution of the pointer designated by ‘Pn’
- The number of repetitions remaining at the point that the FOR to NEXT loop was brought to a forced end is stored at (D)



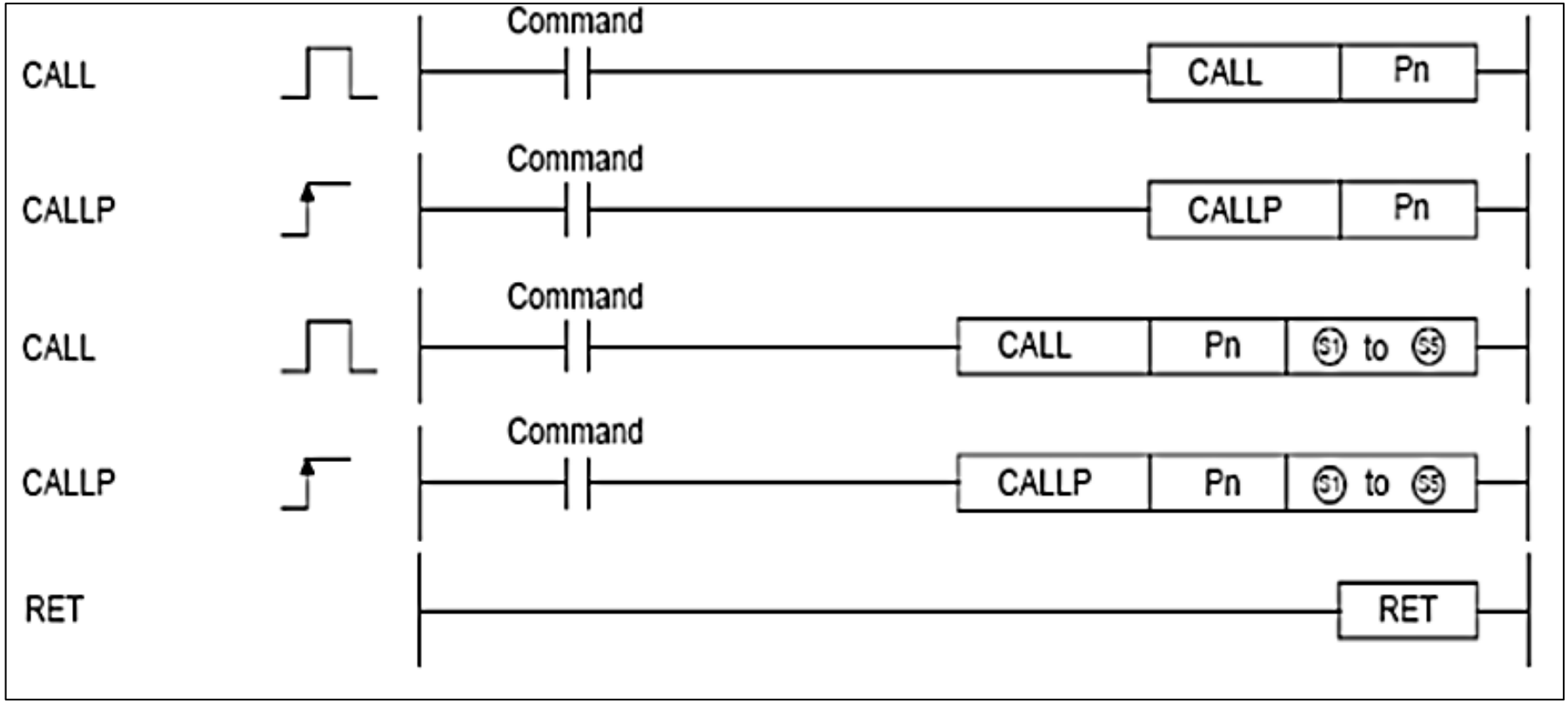
Subroutines

- Sequence program classifications
  - Main routine program
  - Subroutine program
  - Interrupt program
- Subroutine program is called from main program using CALL instruction
- Main program is terminated using FEND instruction
- Subroutine body is written between FEND and END instruction
- Multiple subroutines, interrupt subroutines are allowed.



Subroutine Program Instructions

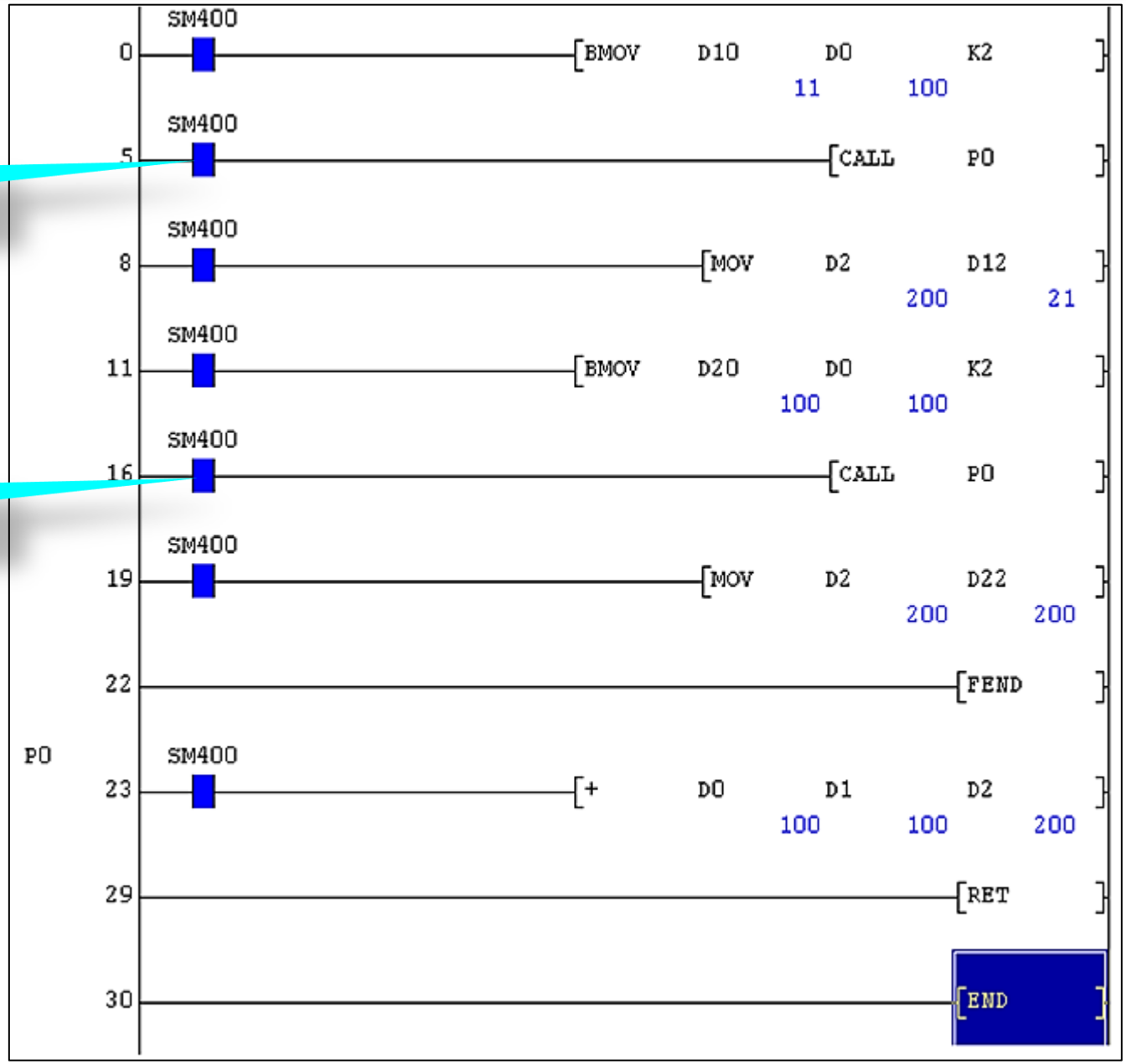
- CALL and CALLP
- RET instructions



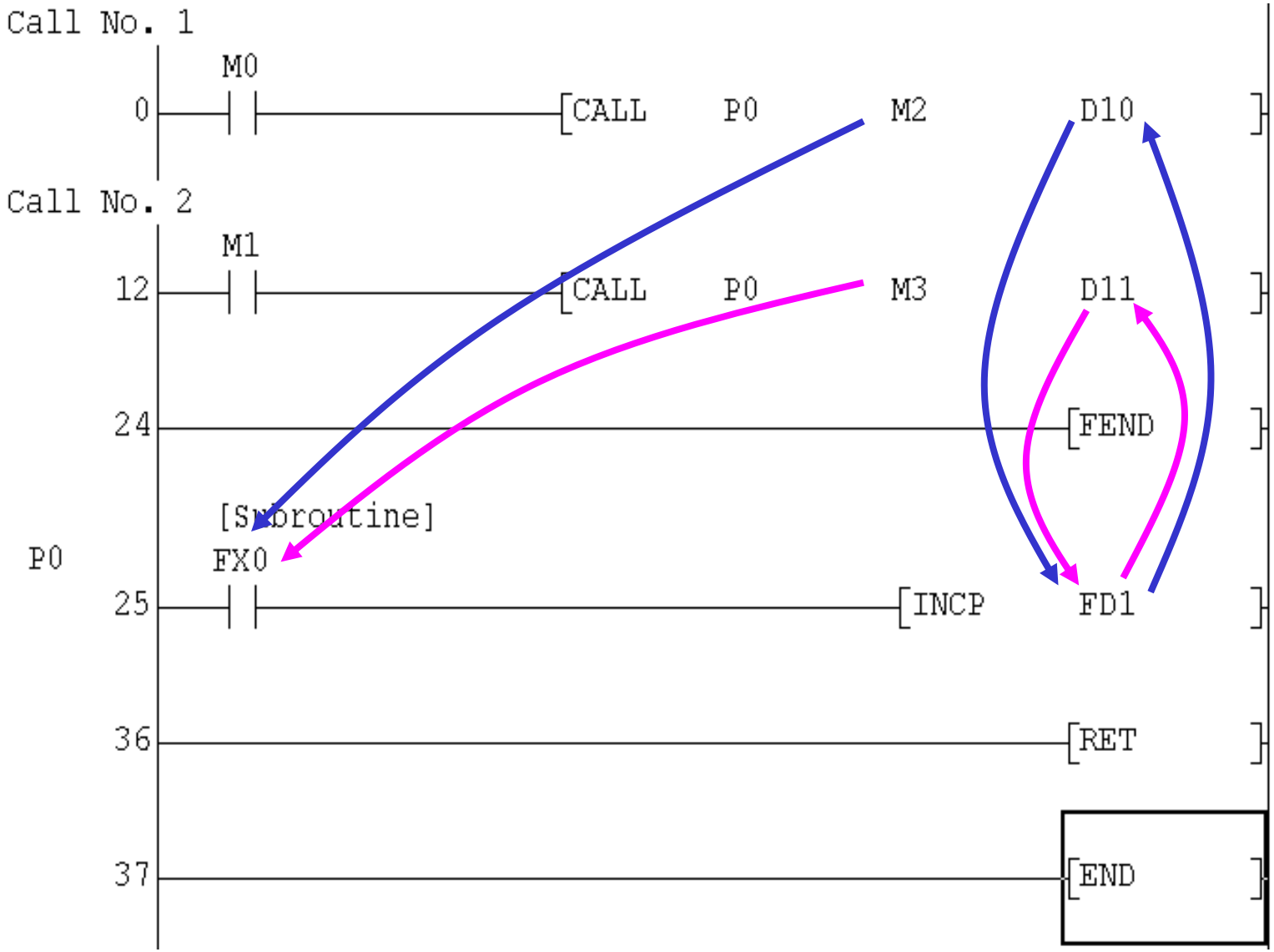
Example of CALL

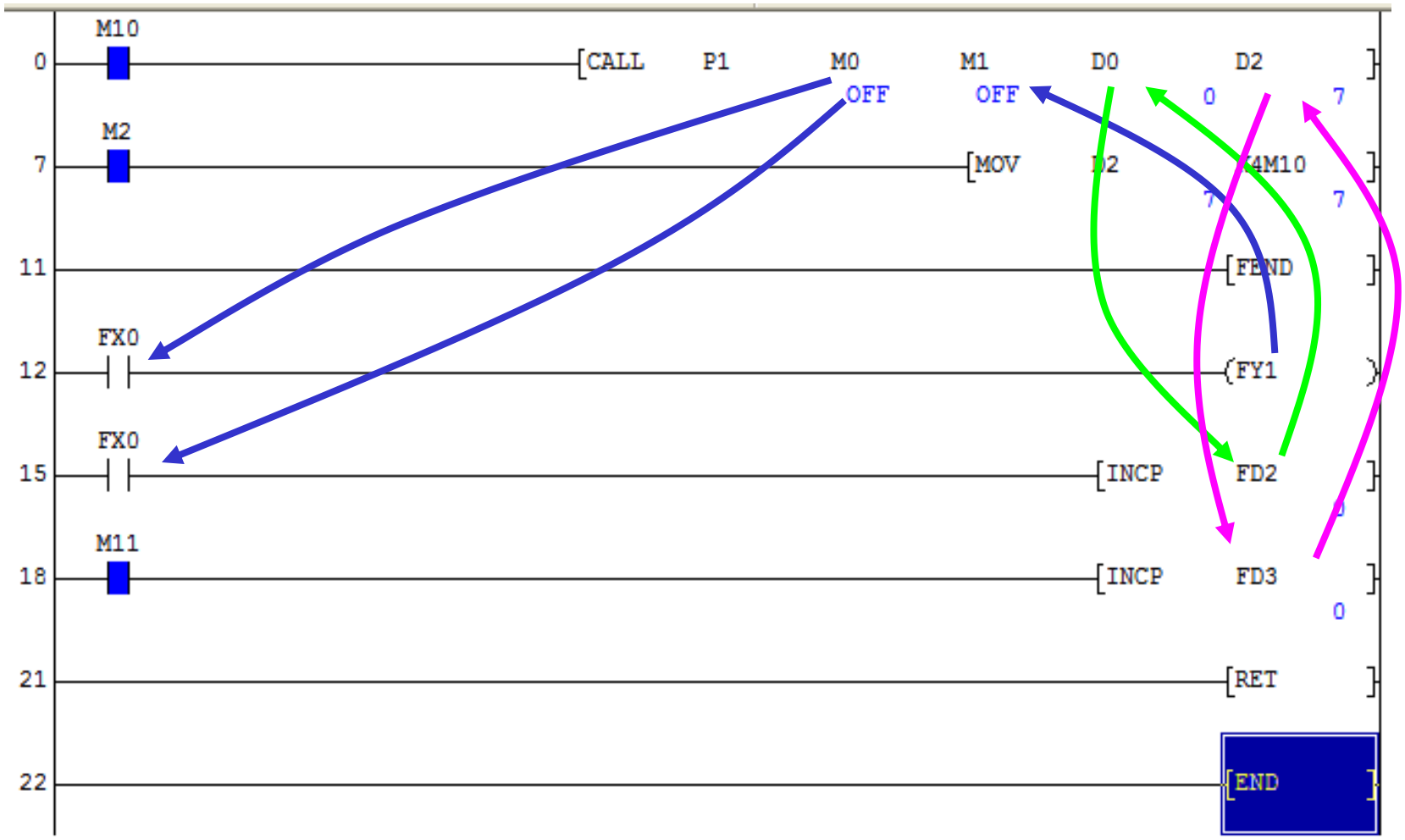
Call 1

Call 2



Example of CALL with Function Devices



Subroutine Example

- -[CALL P1 FX0 FY1 FD2 FD3]